

INFORME TÉCNICO DEL PROYECTO

SIS-Control

Sistema de Control de Rondas para Guardias de Seguridad

Asignatura	Taller Aplicado de Programación, TPY1101
Sección	004V
Grupo	N° 4
Integrantes y roles	Benjamin Burgos Morales: Backend / Base de datos / QA técnico backend Freddy Sepúlveda Vásquez: Mobile / Scrum Master / Documentación funcional David Trureo Ahumada: Backend / Base de datos / QA técnico de datos
Docente	Félix Cifuentes Cid
Fecha	25 de Mayo de 2026.-

Resumen

SIS-Control es una solución de software orientada a digitalizar y controlar rondas de guardias de seguridad mediante una aplicación móvil Android y un backend REST. El sistema busca reemplazar o complementar registros manuales mediante trazabilidad digital de usuarios, instalaciones, checkpoints, rondas, ubicación y eventos operativos asociados al cumplimiento de las labores de vigilancia.

La evidencia revisada permite sostener que el proyecto se encuentra organizado en tres líneas principales: documentación y evidencias del proyecto; aplicación móvil Android en Kotlin; y backend Java/Spring Boot con persistencia MySQL. La integración móvil-backend no queda solo declarada: en la rama Vito/Conección_Backend se observa configuración de Retrofit, OkHttpClient, servicios de API y endpoints REST concretos para autenticación, usuarios, instalaciones y checkpoints.

El servidor previsto para el MVP se documenta como entorno local. En Android, el backend local se consume mediante la URL base `http://10.0.2.2:8080/`, que corresponde al alias usado por el emulador Android para acceder al localhost del equipo anfitrión. En backend, la configuración disponible declara conexión JDBC a MySQL local mediante la base `sis_control`, usuario `root` y puerto `3306`.

Índice

Contenido

RESUMEN	2
ÍNDICE	3
1. INTRODUCCIÓN	4
2. DESCRIPCIÓN GENERAL DEL PROYECTO	4
3. METODOLOGÍA DE DESARROLLO	6
4. PATRONES DE DISEÑO.....	6
4.1 MVVM EN APLICACIÓN MÓVIL.....	6
4.2 REPOSITORY PATTERN.....	6
4.3 INYECCIÓN DE DEPENDENCIAS MANUAL	7
4.4 ARQUITECTURA POR CAPAS EN BACKEND	7
5. ARQUITECTURA DEL SISTEMA.....	7
6. LENGUAJES DE PROGRAMACIÓN Y TECNOLOGÍAS	10
7. CONFIGURACIÓN DE SERVIDOR Y ENTORNO LOCAL.....	11
8. ASPECTOS DE INTEGRACIÓN NECESARIOS.....	15
8.1 INTEGRACIÓN ANDROID → BACKEND	15
8.2 INTEGRACIÓN BACKEND → BASE DE DATOS	16
8.3 INTEGRACIÓN POR ROLES.....	16
8.4 INTEGRACIÓN CON RECURSOS DEL DISPOSITIVO	16
9. ESTADO DE INTEGRACIÓN OBSERVADO EN RAMAS	18
10. MOCKUPS Y EVIDENCIAS VISUALES.....	19
11. MODELO DE DATOS Y DOCUMENTACIÓN TÉCNICA.....	25
12. REPOSITORIOS Y RAMAS REVISADAS	26
13. RIESGOS, BRECHAS Y RECOMENDACIONES	27
14. CONCLUSIONES	28
15. REFERENCIAS.....	29

1. Introducción

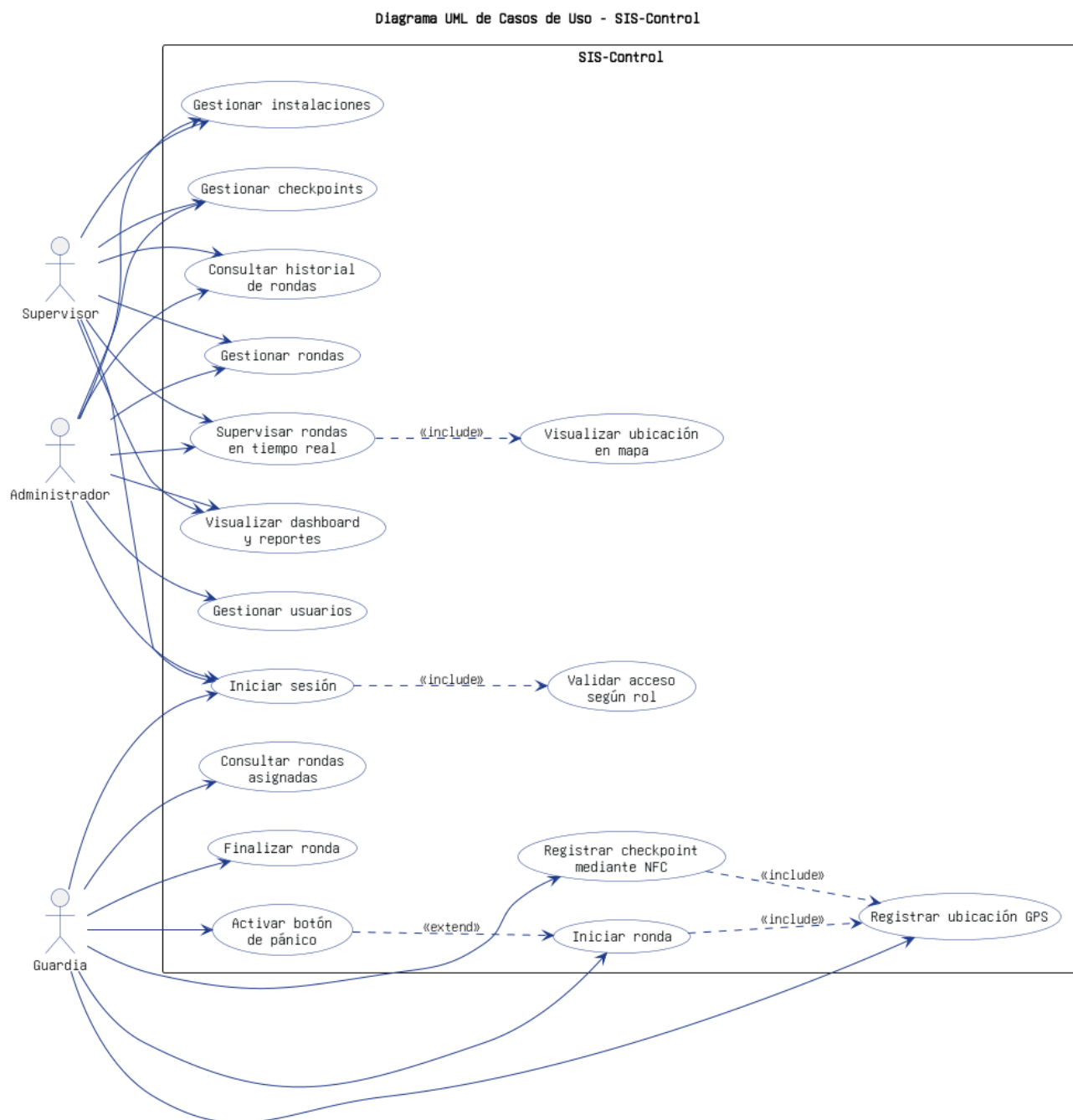
El presente informe describe el proyecto SIS-Control desde una perspectiva técnica, metodológica y documental. El sistema se orienta al control de rondas de guardias de seguridad, incorporando una aplicación móvil Android, un backend REST y una base de datos MySQL para centralizar la información operacional.

La finalidad del documento es consolidar la información del proyecto en un formato formal, apto para entrega académica y revisión técnica. Para evitar afirmaciones no verificadas, el informe distingue entre elementos confirmados en los repositorios y elementos que corresponden a diseño, alcance previsto o evidencia documental disponible en carpetas específicas.

2. Descripción general del proyecto

SIS-Control aborda la necesidad de registrar, supervisar y validar rondas de seguridad. En vez de depender únicamente de registros manuales, el sistema propone trazabilidad digital mediante usuarios con roles diferenciados, gestión de instalaciones, checkpoints y consumo de servicios backend.

Rol	Responsabilidad principal	Funcionalidades asociadas
Administrador	Gestión superior del sistema.	Administrar usuarios, instalaciones, checkpoints, supervisión general y validaciones críticas.
Supervisor	Seguimiento operativo.	Monitorear rondas, revisar registros, apoyar gestión de guardias y consultar evidencia.
Guardia	Ejecución de rondas.	Iniciar rondas, marcar checkpoints y registrar eventos en terreno.



El diagrama de casos de uso permite visualizar las principales interacciones entre los actores del sistema y las funcionalidades previstas para el MVP. En este caso, se identifican los perfiles administrador, supervisor y guardia, cada uno asociado a responsabilidades diferenciadas dentro del control de rondas.

3. Metodología de desarrollo

El proyecto se documenta bajo un enfoque ágil basado en Scrum, adaptado al contexto académico. Este enfoque permite organizar avances por iteraciones, priorizar funcionalidades, construir evidencia incremental y mantener alineación entre planificación, diseño, implementación y validación.

Elemento Scrum	Aplicación en SIS-Control	Evidencia esperada	Observación crítica
Product Backlog	Listado priorizado de historias de usuario y requisitos funcionales.	Carpeta HU_RF_Tareas y documentación de gestión.	Debe mantenerse sincronizado con lo realmente implementado.
Sprint Planning	Planificación de actividades por fase o sprint.	Gantt y planificación semestral.	Debe reflejar responsables reales y no solo planificación ideal.
Incremento	Avance funcional de app móvil y backend.	Commits y ramas activas.	Debe integrarse en ramas principales antes de la entrega.
Review / Validación	Revisión de funcionalidades y mockups.	Capturas, mockups, pruebas y demo.	La demo debe depender de datos reales o endpoints funcionales cuando sea posible.

4. Patrones de diseño

4.1 MVVM en aplicación móvil

La aplicación móvil sigue una organización compatible con MVVM y Clean Architecture: se identifican carpetas de presentación, dominio, datos, repositorios, casos de uso y ViewModels. Esta estructura reduce acoplamiento entre la interfaz Jetpack Compose y la lógica de negocio.

4.2 Repository Pattern

El patrón Repository permite encapsular el acceso a datos remotos. En la rama de integración se observan implementaciones de repositorio que conectan servicios Retrofit con casos de uso del dominio. Esto evita que las pantallas dependan directamente de endpoints o detalles HTTP.

4.3 Inyección de dependencias manual

La aplicación utiliza un módulo AppModule como punto central de ensamblaje. El propio archivo documenta la cadena Retrofit → ApiService → RepositoryImpl → UseCase → ViewModel, lo que evidencia una inyección de dependencias manual, sin Hilt ni Dagger.

4.4 Arquitectura por capas en backend

El backend se estructura de forma coherente con proyectos Spring Boot: controladores REST, servicios, repositorios JPA, entidades/modelos y DTOs. Esta separación permite mantener reglas de negocio fuera de los controladores y persistencia fuera de la interfaz móvil.

5. Arquitectura del sistema

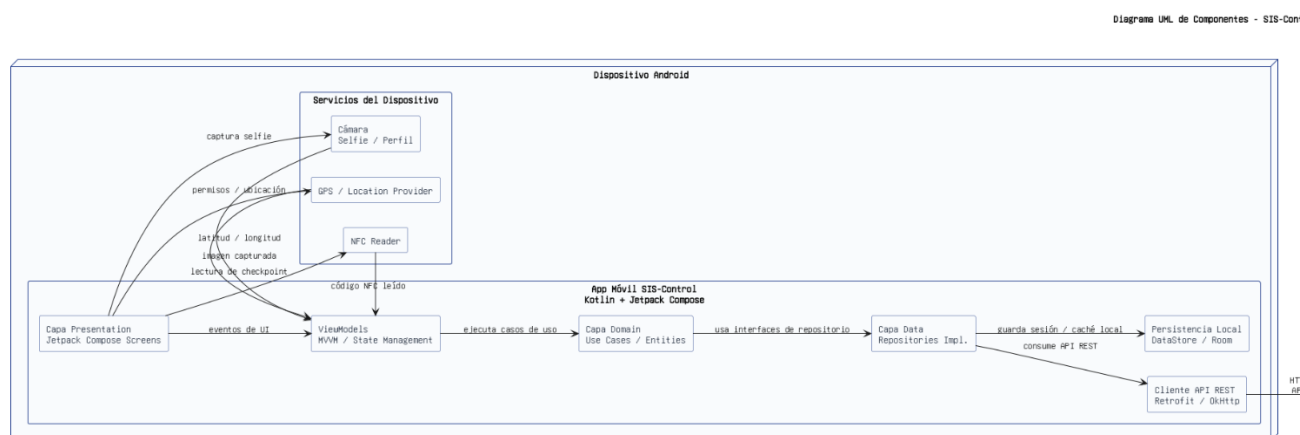
La arquitectura general corresponde a un modelo cliente-servidor con aplicación móvil Android, API REST Spring Boot y base de datos MySQL local para el MVP.

Componente	Descripción técnica
Aplicación Android	Cliente móvil desarrollado en Kotlin. Consume endpoints REST mediante Retrofit y OkHttpClient.
Capa de presentación	Pantallas y ViewModels responsables de estado, navegación y eventos de UI.
Capa de dominio	Casos de uso e interfaces de repositorio para aislar lógica de negocio.
Capa de datos	Servicios Retrofit, DTOs e implementaciones concretas de repositorios.
Backend Spring Boot	API REST desarrollada en Java 17 con Spring Boot, Spring Web MVC, Validation y JPA.
Base de datos MySQL	Persistencia local sis_control mediante JDBC y MySQL Connector/J.

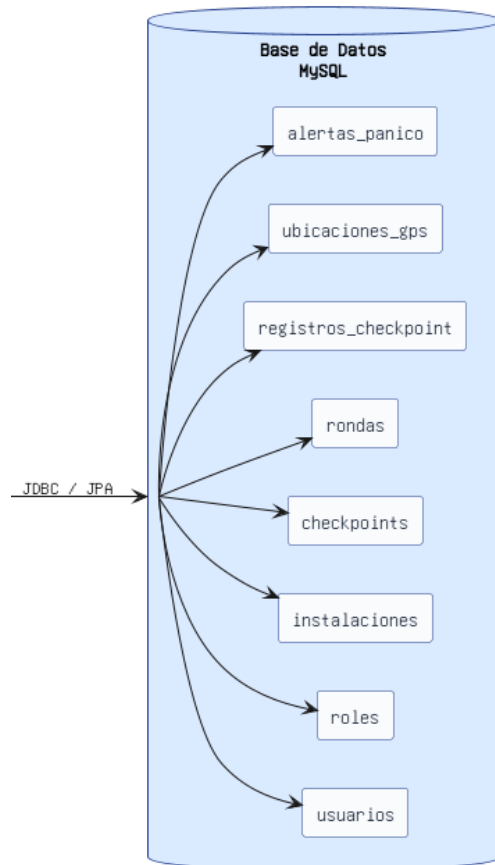
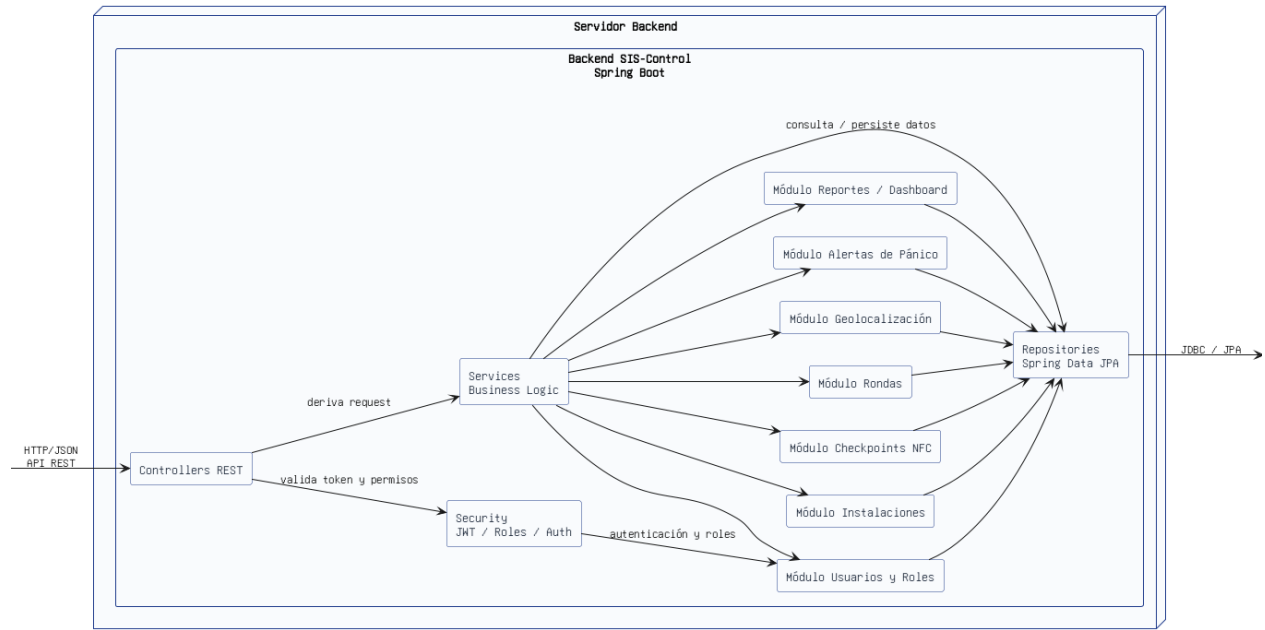
Diagrama textual de arquitectura:

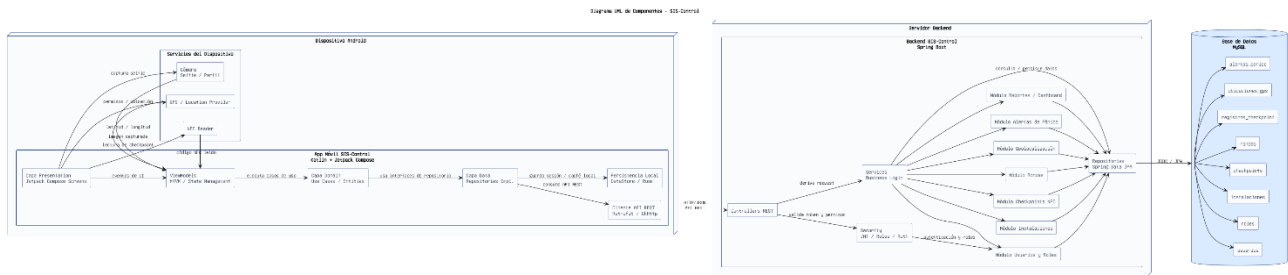
**Usuario móvil → Android App Kotlin/Jetpack Compose → Retrofit/OkHttp →
API REST Spring Boot → JPA/Hibernate → MySQL local**

Como complemento visual, el proyecto cuenta con diagramas UML y documentación técnica en el repositorio documental. Para efectos del presente informe, el flujo anterior resume la arquitectura lógica de comunicación entre cliente móvil, backend y base de datos.



-Control





El diagrama de componentes complementa el flujo textual de arquitectura, mostrando la relación entre la aplicación móvil, los servicios de backend, la capa de persistencia y la base de datos. Este diagrama permite comprender la separación de responsabilidades entre cliente móvil, API REST y almacenamiento de datos.

6. Lenguajes de programación y tecnologías

Área	Tecnología	Uso	Evidencia
Frontend móvil	Kotlin	Desarrollo de app Android.	Repositorio móvil identificado como Kotlin 100%.
UI móvil	Jetpack Compose	Interfaz declarativa y pantallas por rol.	Estructura de presentation y pantallas Compose.
Cliente HTTP	Retrofit + OkHttpClient	Consumo de endpoints REST y manejo de interceptores.	AppModule y ApiService.
Backend	Java 17	Lenguaje del backend.	Propiedad java.version=17 en pom.xml.
Framework backend	Spring Boot 4.0.5	API REST, JPA, validaciones y configuración Maven.	Parent Spring Boot en pom.xml.
Persistencia	MySQL + JPA/Hibernate	Base sis_control local.	application.properties.
Build backend	Maven	Gestión de dependencias y empaquetado.	pom.xml y Maven wrapper.
Build Android	Gradle	Compilación Android.	build.gradle.kts.

7. Configuración de servidor y entorno local

Para este informe, el servidor previsto del MVP queda definido como entorno local. Esta decisión es consistente con la configuración observada en backend y móvil: la API corre en el equipo del desarrollador y la app Android, desde emulador, la consume mediante el alias 10.0.2.2.

Elemento	Configuración documentada	Implicancia
Backend local	Spring Boot en puerto 8080.	La app Android espera acceder a <code>http://10.0.2.2:8080/</code> .
Base de datos	<code>jdbc:mysql://localhost:3306/sis_control</code>	MySQL debe estar iniciado localmente antes de ejecutar backend.
Usuario BBDD	root sin contraseña en configuración actual.	Adecuado para pruebas locales; no recomendable para producción.
JPA/Hibernate	<code>ddl-auto=update</code>	Facilita evolución local del esquema; debe revisarse antes de producción.
Android Manifest	<code>INTERNET</code> y <code>usesCleartextTraffic=true</code>	Permite HTTP local; no es configuración final segura para producción.

7.1 Procedimiento de configuración del ambiente local de pruebas

Para esta etapa del MVP, se definió el ambiente de pruebas en entorno local, ya que permite validar de forma controlada la comunicación entre la aplicación móvil Android, el backend Spring Boot y la base de datos MySQL. Esta configuración resulta adecuada para el estado actual del proyecto, porque facilita las pruebas de integración sin depender todavía de un servidor externo.

El procedimiento de configuración y validación del ambiente local contempla los siguientes pasos:

1. Iniciar el servicio MySQL y validar la creación automática de la base de datos

Se debe iniciar el servicio MySQL antes de ejecutar el backend. En este proyecto, la base de datos `sis_control` se crea automáticamente al iniciar el backend Spring Boot, de acuerdo con la configuración definida en el archivo `application.properties`.

Por lo tanto, no es necesario crear manualmente la base de datos desde MySQL Workbench, phpMyAdmin o consola SQL, salvo que exista un problema de permisos, conexión o configuración.

La validación recomendada consiste en:

- Iniciar MySQL.
- Ejecutar el backend Spring Boot.
- Verificar que el backend inicie sin errores de conexión JDBC.
- Revisar en MySQL que la base `sis_control` haya sido creada correctamente.
- Confirmar que las tablas esperadas se hayan generado o actualizado según la configuración JPA/Hibernate.

En caso de que la base no se cree automáticamente, se debe revisar la URL JDBC, usuario, contraseña, permisos de MySQL, configuración JPA/Hibernate y logs de inicio del backend. Solo como medida de contingencia, puede crearse manualmente la base mediante el siguiente comando:

```
CREATE DATABASE sis_control;
```

2. Configurar la conexión del backend

En el archivo `application.properties` del backend se debe verificar que la conexión JDBC apunte correctamente a la base local:

```
spring.datasource.url=jdbc:mysql://localhost:3306/sis_control
spring.datasource.username=root
spring.datasource.password=
spring.jpa.hibernate.ddl-auto=update
```

Esta configuración es válida para pruebas locales. Sin embargo, para un ambiente productivo o compartido no se recomienda utilizar el usuario root sin contraseña ni mantener credenciales directamente escritas en el archivo de configuración.

3. Ejecutar el backend Spring Boot

Una vez configurada la conexión, el backend debe ejecutarse localmente en el puerto 8080. Durante el inicio, se debe revisar la consola para confirmar que no existan errores de conexión con MySQL, errores de inicialización de dependencias o fallas en la creación/actualización de tablas.

4. Configurar la aplicación Android

Desde el emulador Android, la aplicación debe consumir el backend local mediante la siguiente URL:

http://10.0.2.2:8080/

Esta dirección se utiliza porque 10.0.2.2 corresponde al alias que usa el emulador Android para acceder al localhost del equipo anfitrión.

5. Validar permisos de red en Android

El archivo AndroidManifest.xml debe incluir el permiso de internet:

<uses-permission android:name="android.permission.INTERNET" />

Además, debido a que el MVP se ejecuta en entorno local mediante HTTP, se debe permitir tráfico claro durante las pruebas. Esta configuración es aceptable para el ambiente local, pero no debería mantenerse igual en un despliegue productivo.

6. Ejecutar pruebas de conexión

Finalmente, se recomienda validar primero los endpoints desde Postman y luego desde el emulador Android. De esta manera, es posible diferenciar si un error proviene del backend, de la base de datos, de la red local o de la configuración de la aplicación móvil.

7.2 Procedimiento de respaldo y restauración de base de datos para pruebas

Aunque el MVP se ejecuta en entorno local y todavía no cuenta con un servidor productivo formal, resulta necesario documentar un procedimiento de respaldo y restauración de la base de datos. Esto permite aplicar buenas prácticas de continuidad, recuperación y preparación de ambientes de prueba.

Para esta etapa, la base de datos local *sis_control* puede considerarse como la base principal de trabajo. En caso de requerir pruebas controladas sin afectar los datos principales, puede restaurarse un respaldo en una base separada, por ejemplo *sis_control_test*.

El procedimiento recomendado es el siguiente:

1. Generar respaldo de la base de datos principal

Desde consola, se puede ejecutar:

```
mysqldump -u root -p sis_control > backup_sis_control.sql
```

Este comando permite generar un archivo .sql con la estructura y los datos de la base *sis_control*.

2. Crear una base de datos de pruebas

En caso de necesitar un ambiente separado para restauración, se puede crear una base de pruebas:

```
CREATE DATABASE sis_control_test;
```

3. Restaurar el respaldo en la base de pruebas

Luego, el respaldo puede restaurarse con el siguiente comando:

```
mysql -u root -p sis_control_test < backup_sis_control.sql
```

4. Validar la restauración

Después de la importación, se debe verificar que las tablas principales existan y contengan registros válidos:

```
USE sis_control_test;  
SHOW TABLES;  
SELECT COUNT(*) FROM users;  
SELECT COUNT(*) FROM installations;
```

Los nombres exactos de las tablas deben ajustarse según el esquema real implementado en el backend.

5. Registrar evidencia del proceso

Para respaldar este procedimiento en el informe, se recomienda incorporar evidencias como:

- Captura del archivo .sql generado.
- Captura de la consola ejecutando mysqldump.
- Captura de la base restaurada en MySQL.
- Captura del listado de tablas.
- Captura de consultas de validación de registros.

Este procedimiento permite demostrar que el proyecto considera mecanismos básicos de respaldo, restauración y preparación de datos para pruebas, aun cuando se encuentre en una etapa local de MVP.

8. Aspectos de integración necesarios

8.1 Integración Android → Backend

La aplicación móvil debe mantener centralizada la URL base del backend, declarar permiso de internet, definir servicios Retrofit por recurso y manejar respuestas HTTP. La rama de integración ya contiene estos elementos, incluyendo inyección de JWT mediante interceptor y manejo de error 401.

8.2 Integración Backend → Base de datos

El backend debe mantener configuración JDBC válida, entidades JPA correctamente mapeadas, repositorios Spring Data y validaciones de entrada. En entorno local, MySQL debe contener o permitir crear la base `sis_control`.

8.3 Integración por roles

La separación de roles debe validarse tanto en frontend como en backend. Ocultar botones en la aplicación no equivale a seguridad: cada endpoint sensible debe verificar permisos antes de ejecutar acciones como crear usuarios, modificar instalaciones o administrar checkpoints.

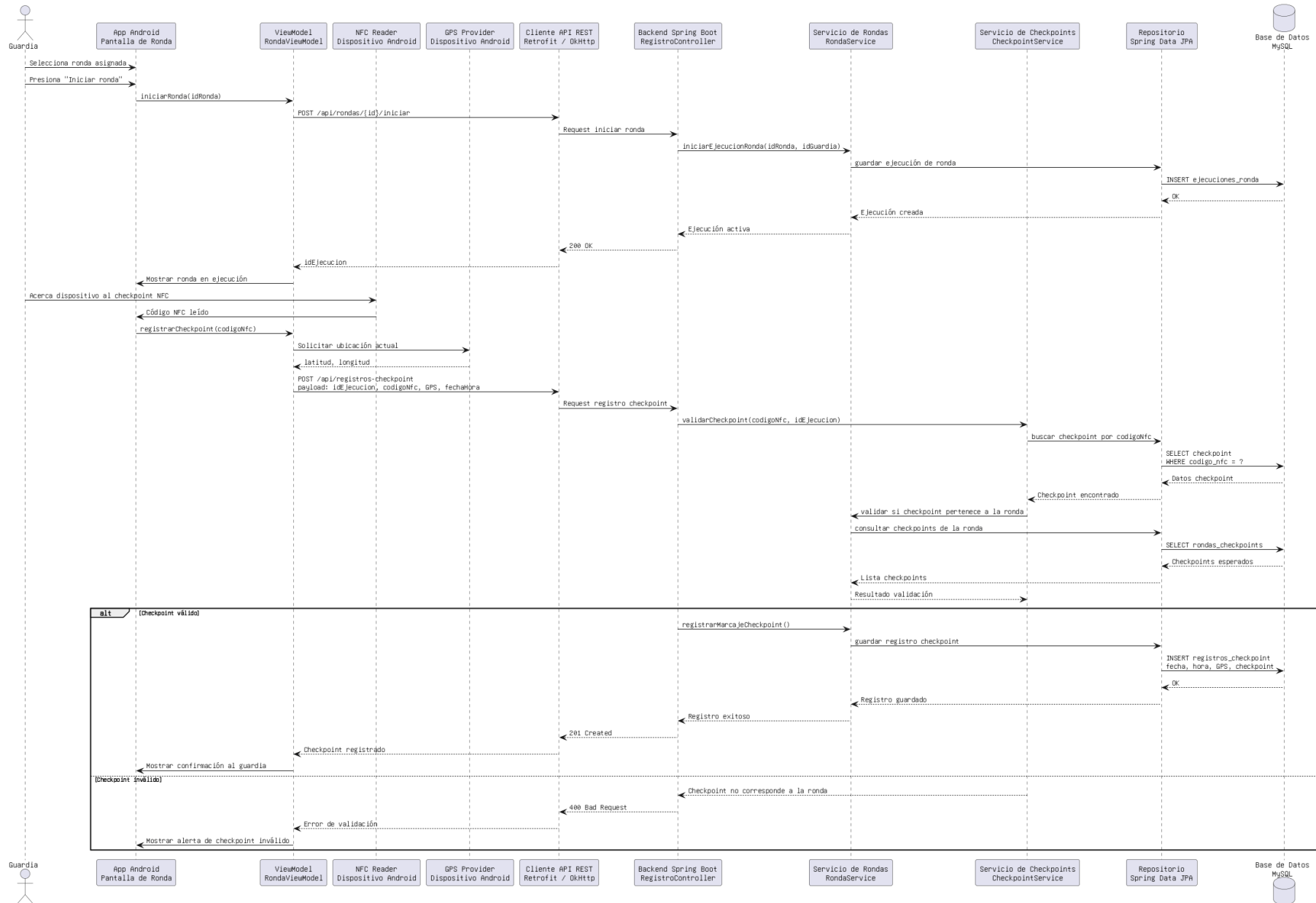
8.4 Integración con recursos del dispositivo

Las funcionalidades asociadas al uso de recursos del dispositivo, como **NFC, GPS y cámara**, ya se encuentran consideradas e integradas dentro del alcance técnico del sistema. Estas funciones son relevantes porque permiten que la aplicación móvil registre evidencia operacional durante la ejecución de rondas, vinculando la acción del guardia con el dispositivo físico utilizado.

Aunque estas funcionalidades forman parte del alcance técnico del MVP, su documentación detallada debe mantenerse como evidencia complementaria. Para ello, se deben registrar formalmente los permisos utilizados, las pantallas asociadas, los flujos de uso y las pruebas realizadas en emulador o dispositivo físico.

En futuras versiones del informe o en anexos técnicos, se recomienda incorporar evidencia específica de validación, especialmente para aquellas funcionalidades que dependen directamente del hardware móvil, como la lectura NFC, el registro de ubicación GPS y el uso de cámara.

Diagrama de Secuencia - Registro de checkpoint NFC y ubicación GPS



El diagrama de secuencia representa el flujo técnico asociado al registro de un checkpoint durante una ronda. Permite visualizar la interacción entre el guardia, la aplicación móvil, los recursos del dispositivo, el backend y la base de datos, evidenciando cómo se registra la marcación junto con información de ubicación y validación.

9. Estado de integración observado en ramas

La rama móvil más relevante para integración es Vito/Conección_Backend. En ella se observa una estructura por capas y consumo real de endpoints REST. No obstante, conviene fusionar o estabilizar esta rama en una rama principal acordada para evitar que la evidencia quede dispersa.

Repositorio	Rama	Hallazgo	Conclusión
Fas73/SIS-Control	master	Rama por defecto actualizada el 4 de mayo de 2026.	No parece ser la rama más actual de integración.
Fas73/SIS-Control	sis-control-david-trureo	Rama activa al 19 de mayo de 2026 con estructura Clean Architecture.	Relevante para trabajo móvil/documental.
Fas73/SIS-Control	Vito/Conección_Backend	Rama activa al 15 de mayo de 2026 con Retrofit, OkHttp, endpoints y BASE_URL local.	Rama principal para evidenciar integración frontend-backend.
Fas73/SIS-Control-Backend	main	Backend Java/Spring Boot y configuración MySQL local.	Base estable del backend.
Fas73/SIS-Control-Backend	feature-finalizar-rondaVITO	Rama activa al 19 de mayo de 2026.	Revisar antes de consolidar demo si contiene cierre de ronda.
DavidTrureo/SIS-Control	actualizacion-documentacion	Contiene carpetas Documentacion, Gestion, Producto y Producto_links.	Rama documental recomendada para informe.

10. Mockups y evidencias visuales

Los mockups se encuentran en el repositorio documental <https://github.com/DavidTrureo/SIS-Control.git> , dentro de la carpeta Documentacion/Wireframes_Mockup.

Mockup / pantalla	Propósito
Login	Acceso inicial y autenticación por credenciales.
Dashboard administrador	Gestión general del sistema.
Dashboard supervisor	Monitoreo de rondas y guardias.
Pantalla guardia	Ejecución de ronda y marcación en terreno.
Instalaciones	Administración de recintos.
Checkpoints	Gestión o visualización de puntos de control.
Ronda activa	Seguimiento del recorrido.
Mapa / ubicación	Visualización GPS y trazabilidad.

Mockup de Login

Acceso inicial y autenticación por credenciales.



El mockup muestra la interfaz de usuario para el sistema SIS-Control. En la parte superior, se encuentra el logo de SIS-Control, que consiste en un ícono de un escudo con una barra de gráficos y una flecha verde, seguido del texto "SIS-Control" y el subtítulo "SEGURIDAD • INFORMACIÓN • SUPERVISIÓN". Debajo del logo, se indica "Inicia sesión en tu cuenta".

El formulario de login incluye dos campos de entrada:

- Correo electrónico:** Un campo con un ícono de correo y el texto "correo@ejemplo.com".
- Contraseña:** Un campo con un ícono de candado y siete puntos para ocultar la contraseña.

Debajo de los campos, hay un botón azul con el texto "Iniciar Sesión".

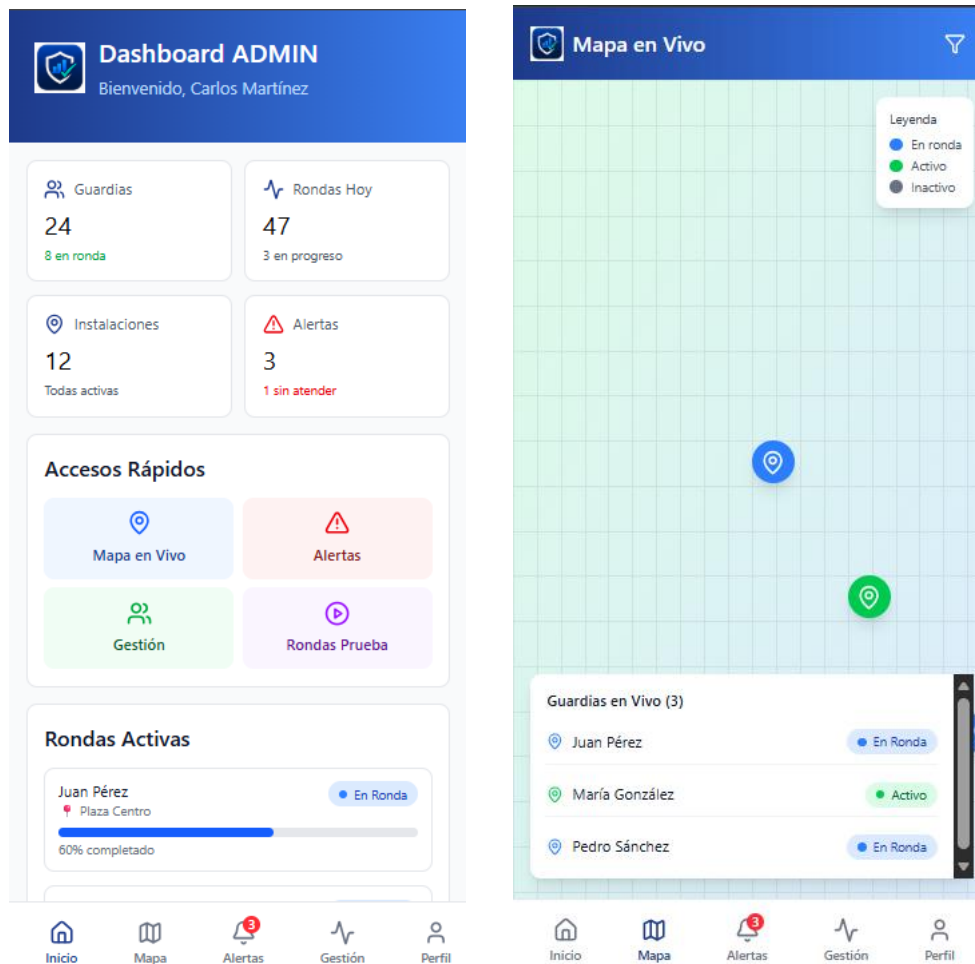
Debajo del botón, se encuentra el enlace "¿Primera vez? Activar cuenta".

En la parte inferior, se indica "Demo: Probar como" seguido de tres botones: "ADMIN", "SUPERVISOR" y "GUARDIA".

Fuente: repositorio documental SIS-Control, carpeta Documentacion/Wireframes_Mockup.

Dashboard Administrador y Seguimiento en vivo

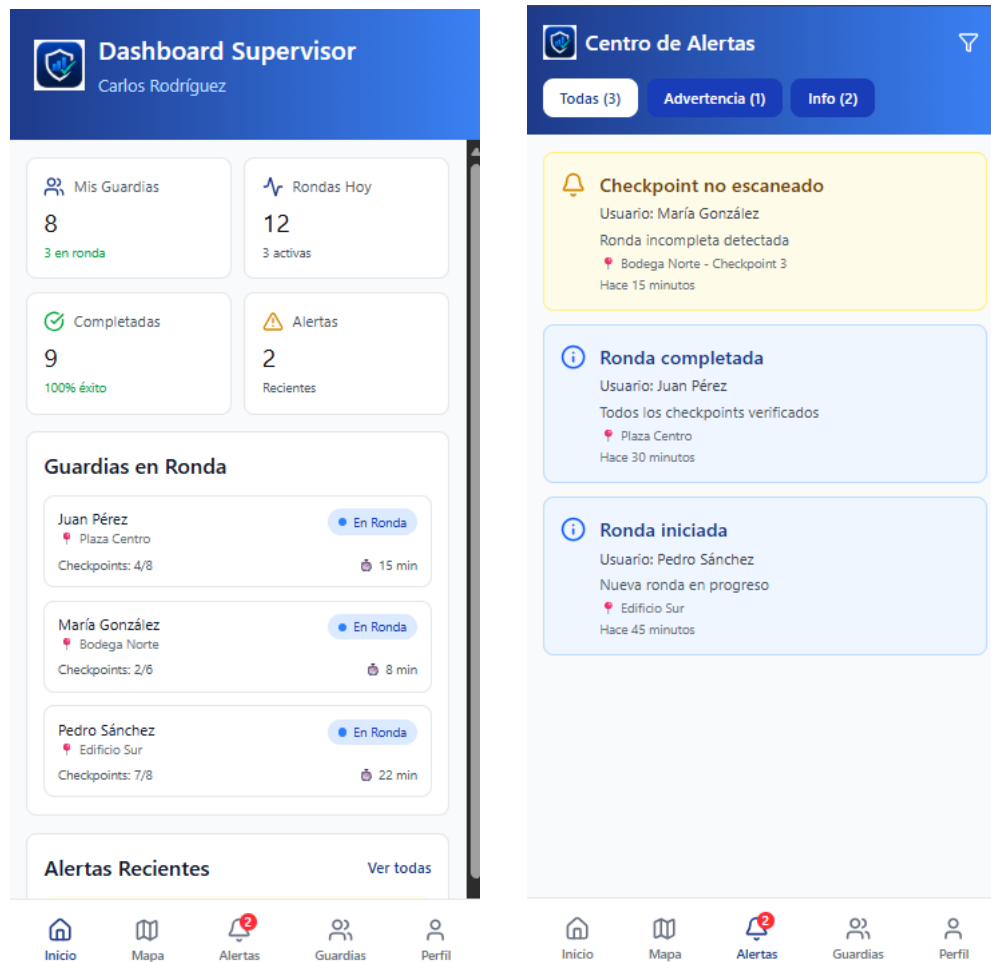
Gestión general del sistema.



Fuente: repositorio documental SIS-Control, carpeta Documentacion/Wireframes_Mockup.

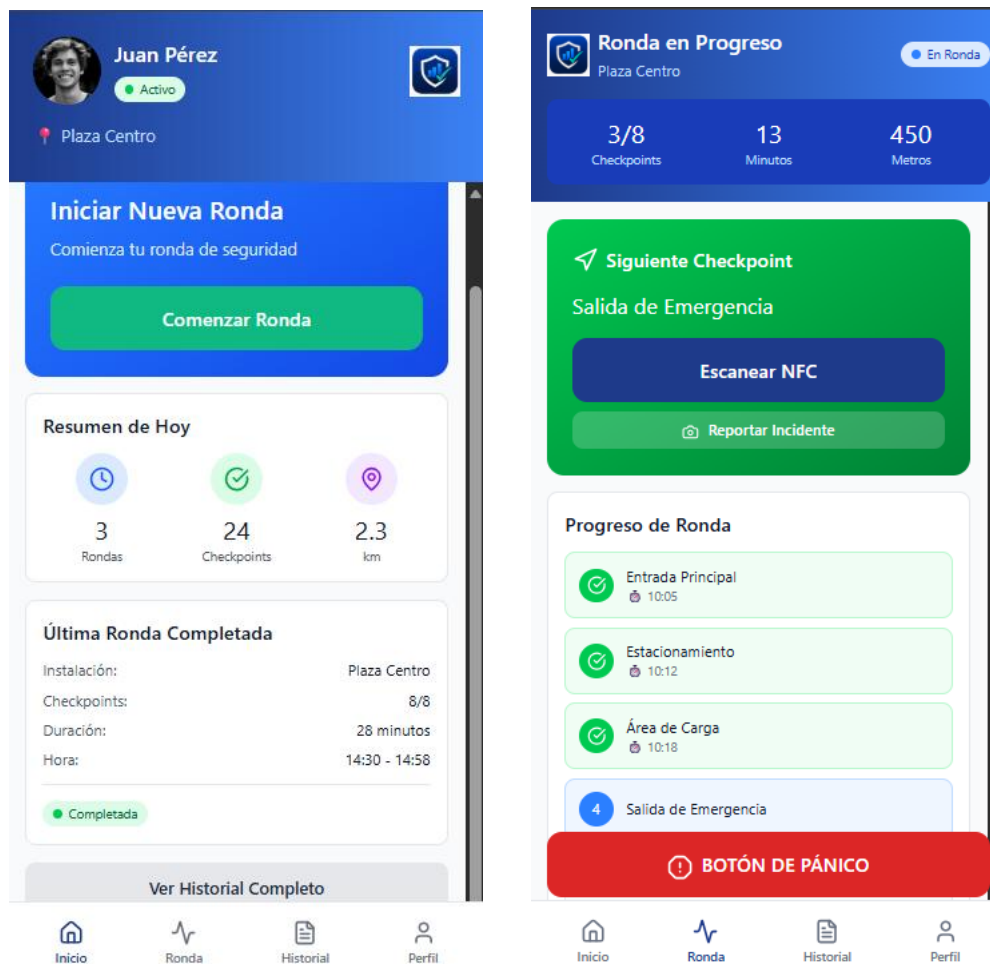
Dashboard Supervisor y pantalla de Alertas

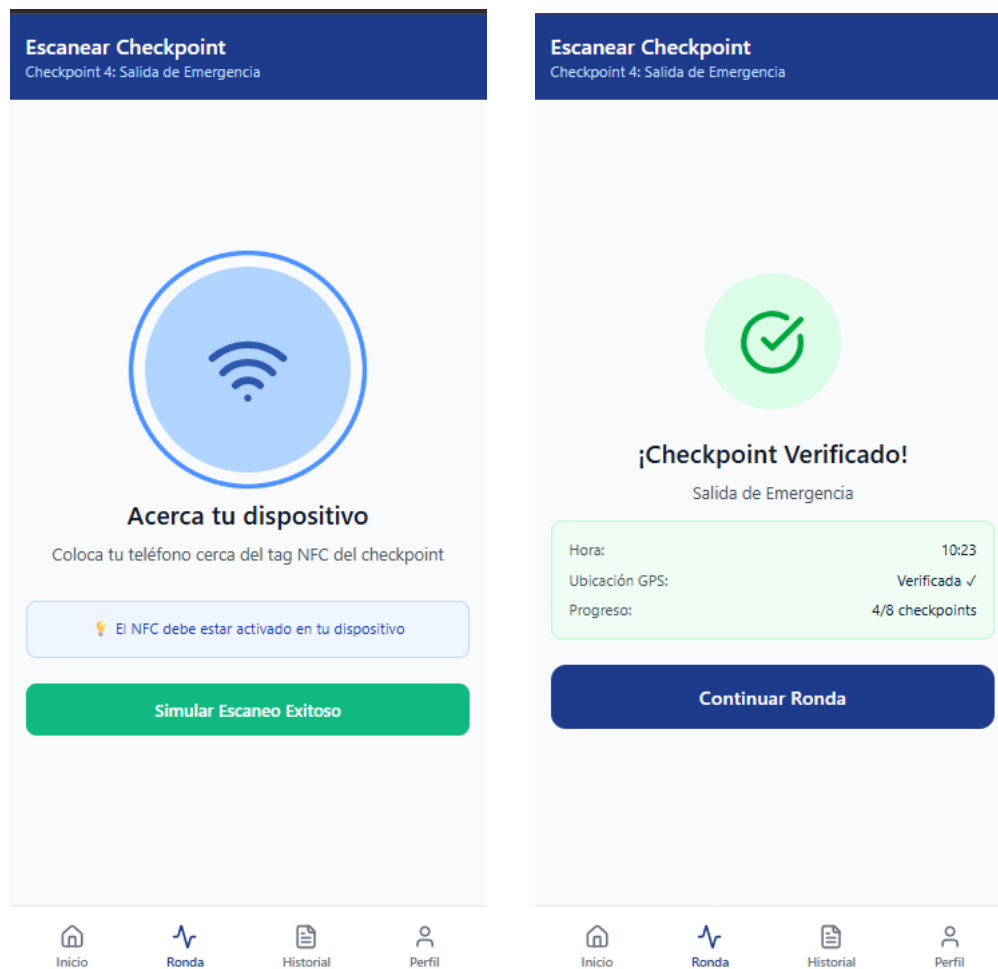
Gestión general del sistema.



Fuente: repositorio documental SIS-Control, carpeta Documentacion/Wireframes_Mockup.

Pantallas inicio guardia y pantallas de procedimiento de escaneo de checkpoints



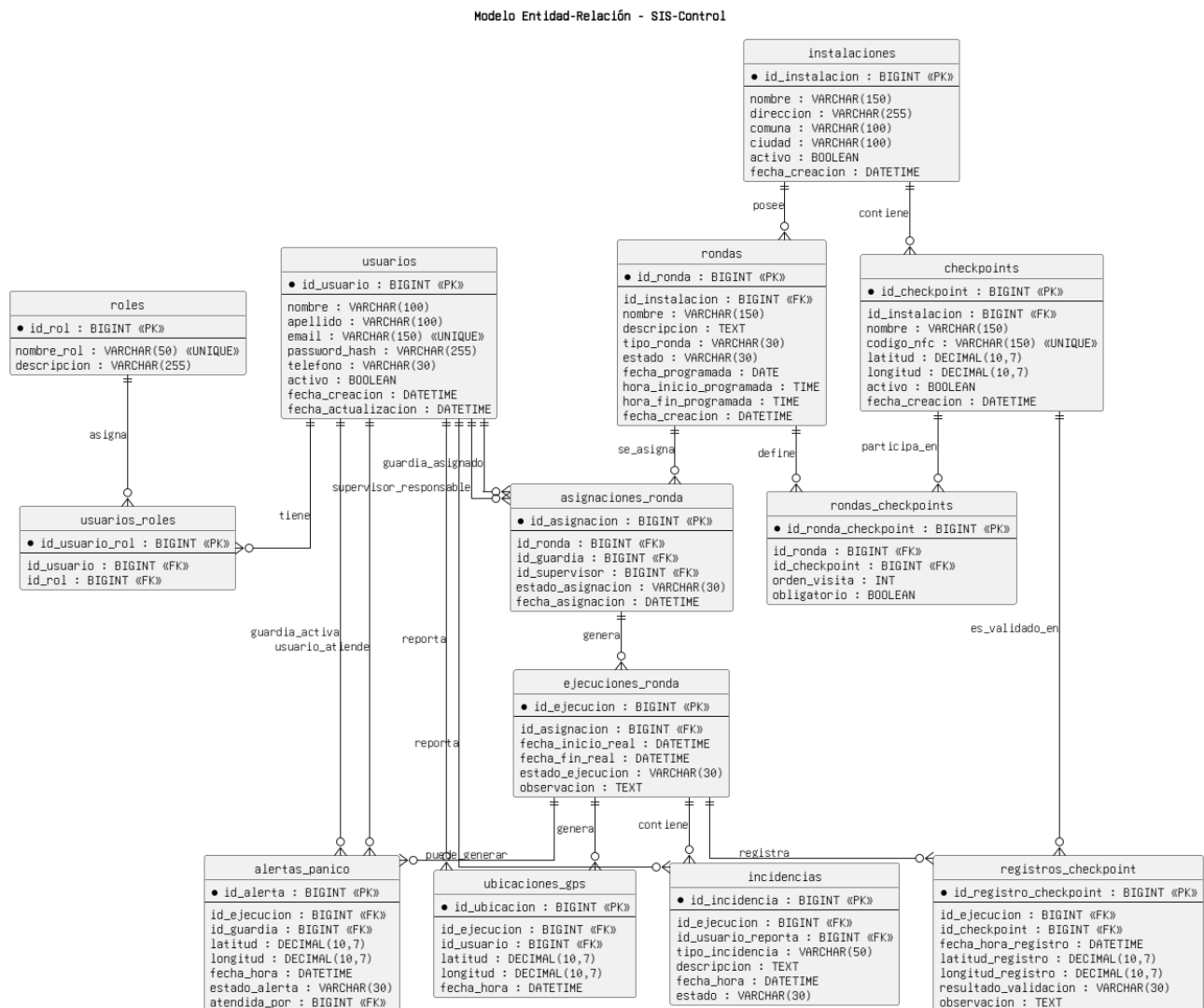


Fuente: repositorio documental SIS-Control, carpeta Documentacion/Wireframes_Mockup.

11. Modelo de datos y documentación técnica

El modelo entidad-relación y la documentación asociada se encuentran en la carpeta Documentacion/MER del repositorio documental: <https://github.com/DavidTrureo/SIS-Control.git>

El diagrama de clases permite representar las principales entidades del sistema y sus relaciones desde una perspectiva orientada a objetos. Su incorporación complementa el modelo de datos, ya que ayuda a vincular las entidades conceptuales del dominio con la estructura lógica utilizada en el desarrollo.



Entidad conceptual	Descripción	Relación esperada
Usuario	Representa usuarios del sistema.	Asociado a uno o más roles según diseño de autenticación.
Rol	Define permisos y nivel de acceso.	Administrador, supervisor y guardia.
Instalación	Recinto o ubicación donde se ejecutan rondas.	Contiene checkpoints.
Checkpoint	Punto de control físico o lógico.	Pertenece a una instalación y se marca durante una ronda.
Ronda	Recorrido planificado o ejecutado por guardia.	Asociada a guardia, instalación y eventos de marcación.
Registro de marcación	Evidencia de paso por checkpoint.	Puede incluir fecha, hora, ubicación, estado y observaciones.

12. Repositorios y ramas revisadas

Repositorio	Uso principal	Observación
DavidTrureo/SIS-Control https://github.com/DavidTrureo/SIS-Control.git	Documentación, gestión, producto, mockups, MER y diagramas UML.	Usar rama actualizacion-documentacion como fuente documental principal.
Fas73/SIS-Control https://github.com/Fas73/SIS-Control.git	Aplicación móvil Android en Kotlin.	Usar rama Vito/Conección_Backend para evidenciar integración backend.
Fas73/SIS-Control-Backend https://github.com/Fas73/SIS-Control-Backend.git	Backend Java/Spring Boot con MySQL.	Usar main como base backend y revisar feature-finalizar-rondaVITO antes de demo.

13. Riesgos, brechas y recomendaciones

Riesgo / brecha	Impacto	Recomendación
Ramas dispersas	La evidencia técnica puede quedar fragmentada.	Definir rama de entrega y fusionar cambios revisados.
Servidor solo local	Demo dependiente del equipo de desarrollo.	Documentar pasos exactos de ejecución local; evaluar túnel temporal solo si se requiere demo desde dispositivo físico.
HTTP claro en Android	Aceptable localmente, no seguro para producción.	Mantener como configuración de MVP local; migrar a HTTPS en despliegue real.
Credenciales MySQL root sin contraseña	Riesgo si se replica fuera de local.	Usar usuario específico y variables de entorno para entornos no locales.
ddl-auto=update	Puede alterar esquema sin control en ambientes compartidos.	Usar migraciones controladas en producción o entrega formal avanzada.
Evidencias visuales no versionadas formalmente	Puede dificultar validar qué mockup, diagrama o captura corresponde a la versión final del informe	Mantener mockups, diagramas, capturas y evidencias en una carpeta versionada del repositorio documental, indicando fecha, versión y fuente

14. Conclusiones

A partir del desarrollo y revisión técnica del proyecto SIS-Control, se concluye que la solución presenta una base coherente para la construcción de un MVP académico y funcional. La combinación de una aplicación móvil Android desarrollada en Kotlin, un backend Java/Spring Boot, una base de datos MySQL local y una integración REST mediante Retrofit permite estructurar una solución alineada con los objetivos definidos para el control de rondas de seguridad.

La integración entre la aplicación móvil y el backend representa uno de los avances técnicos más relevantes del proyecto, ya que permite superar una etapa meramente prototípica y avanzar hacia una solución conectada con servicios reales. En este sentido, la rama Vito/Conección_Backend constituye una evidencia importante del consumo de endpoints desde la aplicación móvil.

Desde una perspectiva documental, el informe queda respaldado por la incorporación de mockups, diagramas técnicos y evidencias visuales del proyecto. Estos elementos permiten demostrar la relación entre planificación, diseño e implementación, fortaleciendo la trazabilidad del trabajo realizado y la comprensión general de la solución propuesta.

Finalmente, se concluye que la decisión de trabajar con servidor local es adecuada para esta etapa del MVP, ya que permite realizar pruebas controladas de integración entre la aplicación móvil, el backend y la base de datos. No obstante, antes de la entrega final es recomendable consolidar las ramas de trabajo en una versión estable, de modo que el frontend integrado, el backend ejecutable y la documentación actualizada queden alineados en una misma evidencia técnica.

15. Referencias

- Repositorio Frontend: <https://github.com/Fas73/SIS-Control.git>
- Repositorio Backend: <https://github.com/Fas73/SIS-Control-Backend.git>
- Repositorio Documentación: <https://github.com/DavidTrureo/SIS-Control.git>
- DavidTrureo/SIS-Control, rama actualizacion-documentacion. Repositorio documental del proyecto.
- DavidTrureo/SIS-Control, carpeta Documentacion. Evidencias de Gantt, HU/RF/Tareas, MER, UML y Wireframes_Mockup.
- Fas73/SIS-Control, ramas master, sis-control-david-trureo y Vito/Conección_Backend. Repositorio de aplicación móvil Android.
- Fas73/SIS-Control, AppModule.kt en rama Vito/Conección_Backend. Configuración Retrofit, OkHttpClient, BASE_URL, repositorios, casos de uso y ViewModels.
- Fas73/SIS-Control, AuthApiService.kt, InstallationApiService.kt y PersonnelApiService.kt en rama Vito/Conección_Backend. Endpoints REST consumidos desde Android.
- Fas73/SIS-Control, AndroidManifest.xml en rama Vito/Conección_Backend. Permiso INTERNET y configuración de tráfico HTTP para entorno local.
- Fas73/SIS-Control-Backend, pom.xml en main. Spring Boot, Java 17, JPA, Validation, Web MVC y MySQL Connector/J.
- Fas73/SIS-Control-Backend, application.properties en main. Configuración local de MySQL y JPA/Hibernate.